

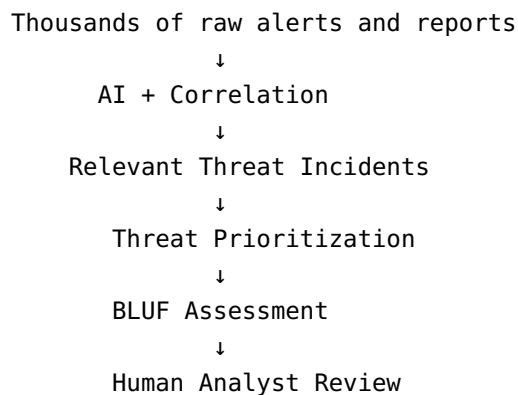
Threat Intelligence Correlation Platform

1. Solution Overview

The proposed solution is an **AI-powered Threat Intelligence Correlation Platform** that collects alerts and intelligence from multiple sources, converts them into a common format, identifies important entities and indicators, correlates related information, prioritizes threats, and automatically generates an evidence-backed **BLUF (Bottom Line Up Front)** assessment.

Core Objective

Transform:



The system is designed to help analysts focus on the **most important threats instead of manually reviewing thousands of individual alerts.**

2. Proposed Solution

2.1 Multi-Source Data Ingestion

The platform accepts information from different sources:

- SIEM alerts
- Cyber sensor events
- OSINT feeds
- Intelligence reports
- PDF documents
- TXT files
- JSON data

- CSV data

Different sources can have completely different formats. The platform first collects and processes these inputs.

Example

SIEM	→ JSON alerts
Cyber Sensor	→ Logs
OSINT	→ Text / API data
Intel Reports	→ PDF / DOCX

3. Data Normalization

After ingestion, all incoming information is converted into a **common internal structure**.

For example:

```
{
  "source": "SIEM",
  "timestamp": "2026-10-06T10:15:00",
  "severity": "HIGH",
  "eventType": "NETWORK_ACTIVITY",
  "description": "Suspicious connection detected",
  "indicators": [
    {
      "type": "IP",
      "value": "xxx.xxx.xxx.xxx"
    }
  ]
}
```

This allows the system to compare information from completely different sources.

4. AI / NLP Processing

The AI layer analyzes text-based intelligence and extracts important information.

Main capabilities

- Entity extraction

- Threat classification
- Information extraction
- Text summarization
- Semantic similarity
- Context analysis

Entities to extract

IP Address
Domain
URL
File Hash
Malware
Threat Actor
Organization
Location
Timestamp
Target
Attack Type

For example:

Input:
"Suspicious traffic was observed from IP X
targeting server ABC."

↓

AI Extraction

IP → X
Target → Server ABC
Event → Suspicious Network Activity

5. Threat Intelligence Correlation

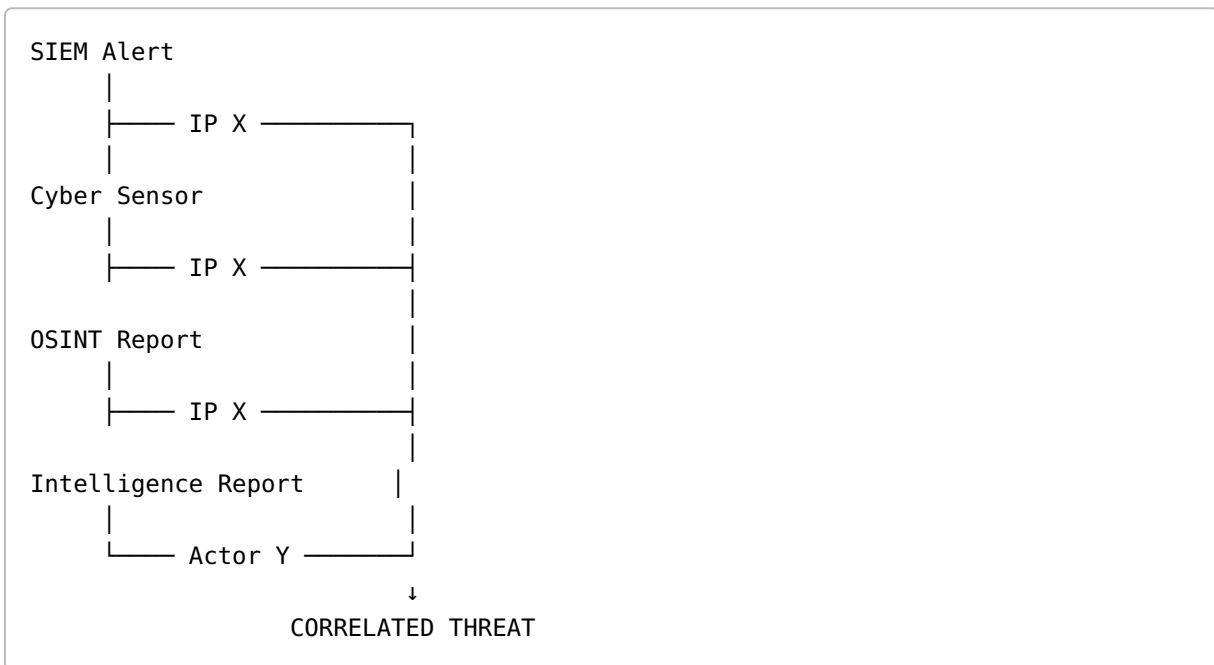
Correlation is the **core functionality** of the platform.

The system identifies relationships between alerts and reports based on:

- Same IP
- Same domain
- Same hash

- Same malware
- Same threat actor
- Same target
- Similar timestamps
- Similar descriptions
- Semantic similarity

Example

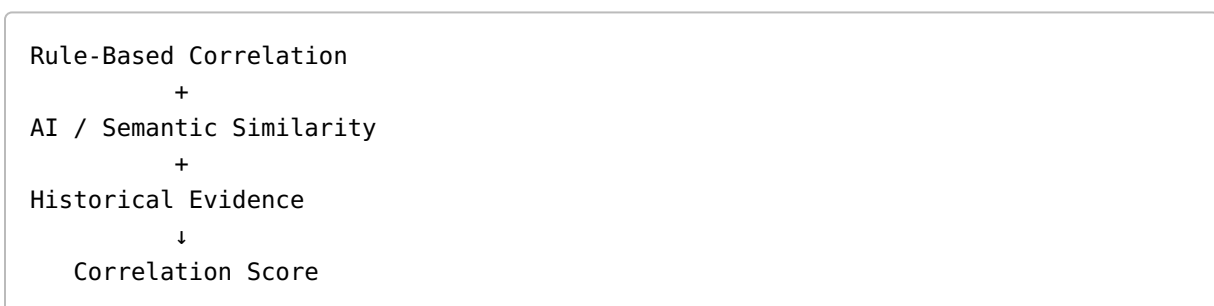


Instead of presenting four independent events, the platform creates a **single correlated threat cluster**.

6. Hybrid Correlation Engine

The platform should not depend entirely on an LLM.

It should combine:



Rule-Based Example

Same IP	→ +30
Same Domain	→ +20
Same Threat Actor	→ +25
Same Target	→ +15
Semantic Similarity	→ +10

The resulting score helps determine whether multiple events are likely related.

This approach is more **explainable and reliable** than allowing an LLM to make the entire decision.

7. Semantic Similarity

Two reports may describe the same activity using different words.

Example:

Report A:
"Malicious communication detected from IP X."
Report B:
"Suspicious network traffic originating from X."

The platform converts the text into embeddings:

Report A → Vector A
Report B → Vector B

The vectors can then be compared to determine semantic similarity.

This helps identify relationships that simple keyword matching may miss.

8. Threat Scoring and Prioritization

After correlation, every threat cluster receives a priority score.

The score can consider:

Severity
+
Source Reliability
+
Correlation Strength
+
Target Criticality
+
Recency
+
Historical Evidence

Example:

0-30 → LOW
31-60 → MEDIUM
61-80 → HIGH
81-100 → CRITICAL

Example Dashboard

Total Alerts: 1,250

CRITICAL → 8
HIGH → 31
MEDIUM → 182
LOW → 1,029

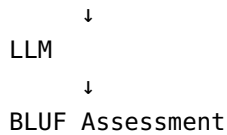
This allows analysts to focus on high-priority incidents first.

9. RAG-Based Evidence Retrieval

RAG (**Retrieval-Augmented Generation**) can be used before generating the final assessment.

Instead of directly asking an LLM to analyze everything:

Raw Reports
↓
Embedding Search
↓
Relevant Evidence



The LLM receives the most relevant evidence instead of relying only on its internal knowledge.

This makes the generated assessment more **evidence-grounded and explainable**.

10. BLUF Generation

The platform automatically generates a structured **Bottom Line Up Front (BLUF)** report.

Example:

```
BLUF

THREAT:
Potential coordinated cyber activity.

CONFIDENCE:
HIGH

AFFECTED TARGET:
Defence infrastructure.

EVIDENCE:
• 12 SIEM alerts
• 4 cyber sensor events
• 2 OSINT reports
• 1 intelligence report

ASSESSMENT:
Multiple independent sources indicate
potential coordinated activity.

PRIORITY:
CRITICAL

ANALYST REVIEW:
REQUIRED
```

The objective is to reduce the time required to produce an initial threat assessment from **hours to minutes**.

11. Human-in-the-Loop

The platform should assist analysts rather than completely replace them.

```
AI Detects
  ↓
AI Correlates
  ↓
AI Prioritizes
  ↓
AI Generates BLUF
  ↓
Human Analyst Reviews
  ↓
Confirm / Reject / Modify
```

Analyst feedback can also be stored and used to improve future prioritization.

12. Technology Stack

Frontend

React + TypeScript + Tailwind CSS

Used for:

- Analyst dashboard
 - Alert management
 - Threat cluster visualization
 - BLUF reports
 - File upload
 - Search and filtering
 - Analyst feedback
 - Threat statistics
-

Backend

Java + Spring Boot

Spring Boot handles:

- REST APIs
 - File uploads
 - Alert management
 - Threat management
 - Correlation services
 - Threat scoring
 - Report generation
 - Authentication
 - Database communication
-

Database

PostgreSQL

Used to store:

```
Users
Alerts
Reports
Indicators
Threat Actors
Threat Clusters
BLUF Reports
Analyst Feedback
```

Vector Search

pgvector

Used with PostgreSQL for:

- Text embeddings
- Semantic similarity
- Similar alert detection
- Similar report detection
- RAG retrieval

This avoids introducing a separate vector database during the initial prototype.

AI / LLM

Ollama + Local LLM

Used for:

- Entity extraction
- Classification
- Summarization
- Threat context analysis
- BLUF generation

A local model can also be useful when working with sensitive or simulated intelligence data.

LLM Integration

Spring AI

Spring AI can connect the Spring Boot backend with the local LLM and provide functionality for:

- Prompt management
 - Structured output
 - Embeddings
 - RAG
 - LLM communication
-

Document Processing

Apache Tika

Used to extract text from documents such as:

PDF
DOCX
TXT

Pipeline:

PDF / Document
↓
Apache Tika
↓
Extracted Text
↓
AI / NLP Processing

Visualization

Recharts

Used for:

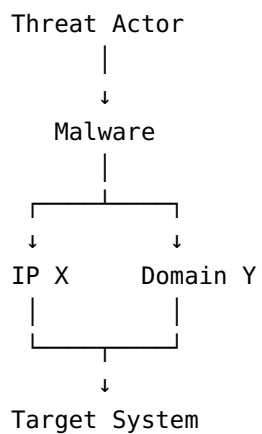
- Threat statistics
- Alert distribution
- Severity charts
- Timeline charts

React Flow

Used for:

- Threat relationship graphs
- Entity relationships
- Correlation visualization

Example:



Authentication

Spring Security + JWT

Used for:

- User authentication
- Role-based access
- Analyst accounts
- Admin accounts

Example:

ADMIN

- └─ Manage users
- └─ Manage system

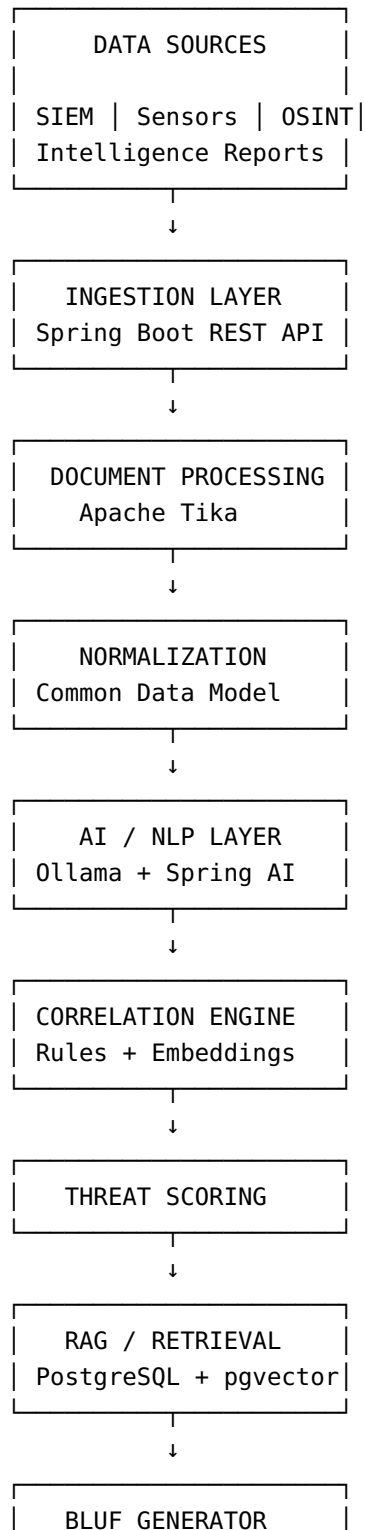
ANALYST

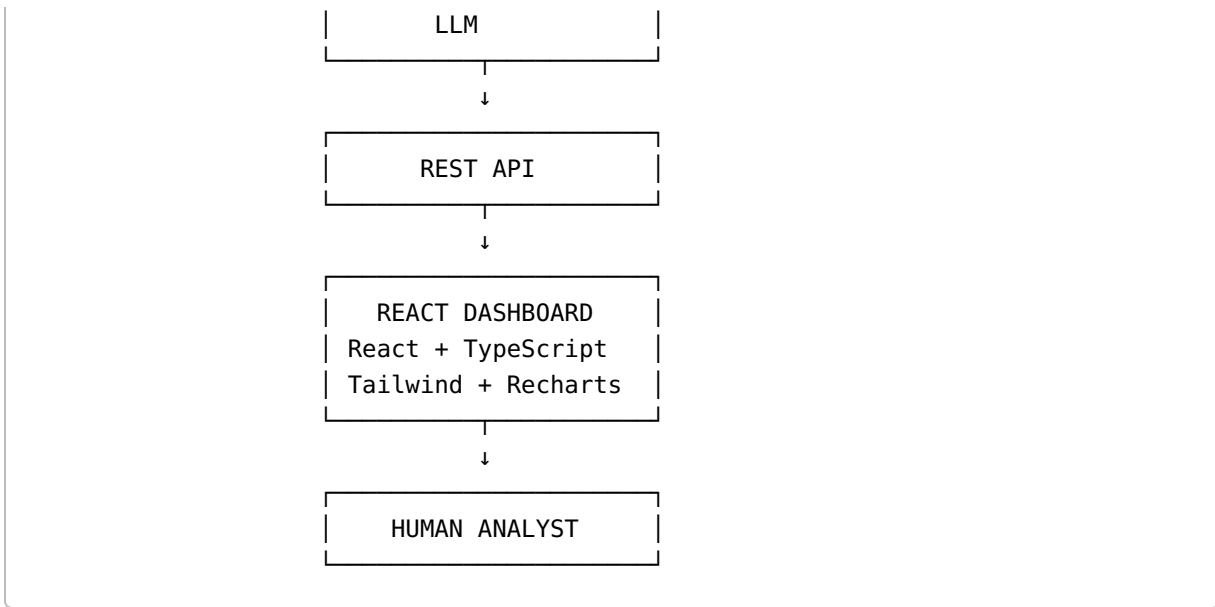
- └─ View alerts
- └─ Analyze threats
- └─ Generate BLUF
- └─ Provide feedback

Development Tools

Purpose	Technology
Backend IDE	Intellij IDEA
Frontend IDE	VS Code
API Testing	Postman
Version Control	Git
Repository	GitHub
Containerization	Docker
Database	PostgreSQL
Vector Search	pgvector
AI Runtime	Ollama
LLM Integration	Spring AI
Document Extraction	Apache Tika

13. Recommended Architecture





14. Technology Summary

Frontend
React + TypeScript + Tailwind CSS

↓

Backend
Java + Spring Boot

↓

Database
PostgreSQL + pgvector

↓

Document Processing
Apache Tika

↓

AI
Ollama + Local LLM

↓

LLM Integration
Spring AI

↓

Correlation
Rules + Semantic Similarity

↓

Retrieval
RAG + pgvector

↓

Output

